

Security Findings & Fix Report — mx-chain-es-indexer-go

Scan Type: Full repository scan — all files analyzed **Directories Covered:** `api/`, `client/`, `cmd/`, `config/`, `core/`, `data/`, `facade/`, `factory/`, `integrationtests/`, `metrics/`, `mock/`, `process/`, `scripts/`, `templates/`, `tools/` **Overall Status:** 13 findings total — 6 real security findings (4 High, 2 Medium), 3 DRWA-specific findings (1 High, 2 Medium), 1 DRWA confirmed safe with hardening note, 1 false positive, 1 code quality finding, 1 DRWA flow integrity gap. The most critical issues are a Painless script injection exploitable by any on-chain NFT creator, two Elasticsearch query injections that can wipe all DRWA compliance indices on block revert and finalize, missing Elasticsearch authentication exposing all KYC/AML/attestation data, and a DRWA design gap where any contract can spoof compliance records by emitting canonical event identifiers.

SECTION 1 — SUMMARY TABLE

#	File	Line	Severity	Type	Fix Required	Status
1	<code>process/elasticprocessor/converter/tokenMetadata.go</code>	160–161	High	Painless Script Injection via On-Chain NFT Attributes	Yes	REAL FINDING
2	<code>process/elasticprocessor/elasticProcessor.go</code>	432–441	High	ES Query Injection — DRWA Revert Delete-by-Query	Yes	REAL FINDING
3	<code>process/elasticprocessor/elasticProcessor.go</code>	443–461	High	ES Query Injection — DRWA Finalize Update-by-Query	Yes	REAL FINDING
4	<code>cmd/elasticindexer/config/prefs.toml</code>	21–23	High	Missing Elasticsearch Authentication	Yes	REAL FINDING
5	<code>api/gin/webServer.go</code>	61–62	High	CORS AllowAllOrigins with Authorization Header	Yes	REAL FINDING
6	<code>tools/accounts-balance-checker/pkg/</code>	48–84	Medium	ES Query Injection via On-Chain Token Identifier	Yes	REAL FINDING

#	File	Line	Severity	Type	Fix Required	Status
	check/ query.go					
7	process/ elasticpr oc/ logsevent s/ drwaEvent sProcesso r.go	88	Medium	No Contract Address Verification for DRWA Events	Yes	DRWA FINDING
8	process/ elasticpr oc/ logsevent s/ drwaEvent sProcesso r.go	288–506	Medium	Unvalidated On-Chain Strings in Compliance Records	Yes	DRWA FINDING
9	process/ elasticpr oc/ logsevent s/ serialize Drwa.go	121–128	Info	Document ID Oversized — Silent Bulk Drop	Recommended	DRWA SAFE / HARDENING
10	docker- compose.y ml	7	Medium	xpack.securi ty Disabled — No Auth, No TLS, No Audit	Yes	REAL FINDING
11	tools/ accounts- balance- checker/ cmd/ balance- checker/ config.js on	3–5	Medium	Plaintext Credential Structure Invites Git Leak	Yes	REAL FINDING
12	api/gin/ httpServe r.go	50	Info	1-Second Graceful Shutdown Drops In- Flight Requests	No	CODE QUALITY
13	process/ factory/ indexerFa ctory.go	110–112	—	Empty Credentials Passed to ES Client — Appears Racy But Is	None	FALSE POSITIVE

#	File	Line	Severity	Type	Fix Required	Status
				Intentional Template		

SECTION 2 — REAL FINDINGS

Finding 1 — REAL FINDING — Painless Script Injection via On-Chain NFT Attributes in `tokenMetadata.go` line 160

Classification: - CWE: CWE-94 (Improper Control of Generation of Code — Code Injection) - CVSS v3.1 Score: **9.8 (High)** — AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H - Severity: **High** - Fix Required: Yes - Runtime Impact: Arbitrary Painless code executes inside Elasticsearch on every NFT attribute update — any indexed document field can be overwritten with attacker-controlled values - Monitoring Impact: Malicious script executions appear as normal update operations in ES logs — no distinguishing marker, no error emitted on success

Severity Note: High because `newMetadata` — extracted verbatim from on-chain NFT attributes — is embedded directly into the Painless script `source` string via `fmt.Sprintf`. `FormatPainlessSource` only strips newlines and tabs; it does not escape quotes or any character that carries meaning inside Painless or its enclosing JSON. The data origin is EXTERNAL and ON-CHAIN — NFT attributes are written by token creators on the MultiversX blockchain. Any token creator, with no special privilege beyond the ability to create an NFT, can craft attributes that break out of the string literal context and inject arbitrary Painless statements. Painless runs inside a sandbox, but even within the sandbox an attacker can overwrite any field on any document being updated — including `balance`, `kycStatus`, `amlStatus`, and `isFinalized` on DRWA compliance records.

What the Vulnerable Function Does:

`PrepareNFTUpdateData` in `tokenMetadata.go` builds Elasticsearch scripted-update payloads for NFT metadata changes. For the attribute-update branch it calls `ExtractMetadataFromAttributes` on the raw on-chain attribute bytes, then embeds the result as a literal string value inside the Painless `source` field using `fmt.Sprintf`. The resulting JSON is sent to Elasticsearch which compiles and executes the Painless source.

What it does NOT do: It does not escape `newMetadata` before embedding it. It does not validate that `newMetadata` contains only safe characters. `FormatPainlessSource` is not a sanitiser — it only removes whitespace.

Call chain: `SaveTransactions` → `indexLogsData` → `indexAlteredAccounts` → `saveAccountsESDT` → `indexAccountsESDT` → `SerializeAccountsESDT` → `PrepareNFTUpdateData` → `fmt.Sprintf` embeds raw on-chain string → Elasticsearch compiles and executes injected Painless.

Where Does the Vulnerable Data Come From:

Token creator submits NFT create/update transaction on-chain → MultiversX VM executes contract → contract writes attributes bytes to NFT metadata trie → `mx-chain-go` node emits `outportBlock` with altered accounts → `WebSocket` delivers payload to indexer → `SaveTransactions` processes the block → `ExtractMetadataFromAttributes` extracts the raw metadata string from attributes bytes → `newMetadata` holds attacker-controlled string → `fmt.Sprintf` embeds it into Painless source → Elasticsearch compiles and executes.

Who Uses This Data and Why It Must Be Trusted:

1. **DevOps (Node Operator):** Monitors Elasticsearch index health and document counts. Breaks if injected scripts silently corrupt document fields — balance fields show attacker-controlled values with no error in ES logs. Why watching matters: a sudden change in indexed account balances with no corresponding transaction is the only signal. Silent failure consequence: operators assume the index is correct; corrupted data propagates to all downstream consumers.
2. **Security (Compliance Engineer):** Relies on indexed DRWA compliance fields (`kycStatus` , `amlStatus` , `isFinalized`) being accurate reflections of on-chain state. Breaks if an injected script overwrites these fields — compliance dashboards show false KYC/AML approvals. Why watching matters: `isFinalized=true` on an unfinalized record bypasses downstream finality checks. Silent failure consequence: regulated transfers are approved based on corrupted compliance data.
3. **Compliance (Regulatory Officer):** Must demonstrate that indexed data accurately reflects on-chain state for regulatory reporting. Breaks if injected scripts alter transaction or account records — audit trail is corrupted. Why watching matters: MiCA and SEC regulations require tamper-evident audit trails. Silent failure consequence: regulatory filing contains attacker-controlled values with no indication of tampering.
4. **On-Call Engineer:** Receives alerts for unexpected field values in indexed documents. Breaks because the injected script executes as a normal update — no error is logged, no metric is emitted. Why watching matters: the only signal is a field value that does not match the expected on-chain state. Silent failure consequence: on-call cannot distinguish a legitimate update from an injected one without replaying the entire block.

What an Attacker Can Do:

1. **Balance Corruption:** Attacker creates an NFT with attributes containing a Painless injection that overwrites the `balance` field on the target document.
 - Crafted input: NFT attributes `metadata:x\", \"tags\": null}} } ctx._source.balance = \"999999999999\"; if (true) {//`
 - Exact log output: No error — Elasticsearch executes the injected script and returns HTTP 200
 - Consequence: Indexed account balance shows attacker-controlled value; downstream balance checkers report incorrect balances.
2. **DRWA Compliance Record Corruption:** Attacker creates an NFT whose attribute update targets a DRWA compliance index document and sets `isFinalized=true` on an unfinalized record.
 - Crafted input: NFT attributes `metadata:x\", \"tags\": null}} } ctx._source.isFinalized = true; if (true) {//`
 - Exact log output: No error — update succeeds silently
 - Consequence: Unfinalized DRWA records are marked finalized; downstream compliance consumers approve transfers that should be pending.
3. **KYC/AML Status Spoofing:** Attacker injects a script that sets `kycStatus` and `amlStatus` to `approved` on a holder compliance record.
 - Crafted input: NFT attributes `metadata:x\", \"tags\": null}} } ctx._source.kycStatus = \"approved\"; ctx._source.amlStatus = \"approved\"; if (true) {//`
 - Exact log output: No error — Elasticsearch executes both assignments

- Consequence: Blocked holders appear KYC/AML approved in the compliance index; regulated transfers proceed for non-compliant holders.

4. **Index-Wide Document Deletion:** Attacker injects `ctx.op = 'delete'` to delete the target document entirely.

- Crafted input: NFT attributes `metadata:x\", \"tags\": null}}` } `ctx.op = 'delete'; if (true) { //`
- Exact log output: No error — document is deleted silently
- Consequence: Indexed NFT token record is permanently deleted; token appears non-existent to all downstream consumers.

Why This Is Specific to This Feature:

`PrepareNFTUpdateData` is the only function in the entire indexer codebase that embeds a user-controlled string (`newMetadata`) directly into a Painless script `source` field. Every other scripted update in the codebase either uses only numeric/boolean parameters (which cannot carry injection payloads) or uses `json.Marshal` for the entire payload. This function sits at the root of the NFT metadata update path — every NFT attribute update on the entire MultiversX blockchain passes through it.

The Fix:

BEFORE:

```
newMetadata := ExtractMetaDataFromAttributes(nftUpdate.NewAttributes)
// ...
serializedData := []byte(fmt.Sprintf(
    `{"script": {"source": "%s", "lang": "painless", "params": {"attributes": "%s", "metadata":
      "%s", "tags": %s}}, "upsert": {}}`,
    FormatPainlessSource(codeToExecute),
    base64Attr,
    newMetadata, // ← raw on-chain string embedded into Painless source
    marshaledTags,
))
```

AFTER:

```
newMetadata := ExtractMetaDataFromAttributes(nftUpdate.NewAttributes)
// ...
// Static script – source never changes, all user data goes through params
const updateScript = `
    if (ctx._source.containsKey('data')) {
        ctx._source.data.attributes = params.attributes;
        if (params.metadata != null && params.metadata != '') {
            ctx._source.data.metadata = params.metadata;
        } else {
            ctx._source.data.remove('metadata');
        }
        if (params.tags != null) {
            ctx._source.data.tags = params.tags;
        } else {
            ctx._source.data.remove('tags');
        }
    }
`
```

```

payload := map[string]interface{}{
    "script": map[string]interface{}{
        "source": FormatPainlessSource(updateScript), // static - never compiled with user data
        "lang":   "painless",
        "params": map[string]interface{}{
            "attributes": base64Attr, // safe - base64 encoded
            "metadata":    newMetadata, // safe - passed as typed param, never compiled
            "tags":        newTags,     // safe - passed as typed param, never compiled
        },
    },
    "upsert": map[string]interface{}{},
}
serializedData, err := json.Marshal(payload)
if err != nil {
    return err
}

```

What each line does: - `const updateScript` — script source is now a compile-time constant; it can never contain user data - `"params": map[string]interface{}{...}` — all on-chain values are passed as typed Painless parameters, not compiled as code - `json.Marshal(payload)` — Go's JSON encoder handles all escaping; no `fmt.Sprintf` string interpolation - Remove `fmt.Sprintf` — eliminates the injection surface entirely

Why This Fix Is Safe: No new imports needed beyond `encoding/json` which is already imported. Runtime impact: Elasticsearch receives identical semantic instructions — only the delivery mechanism changes from string interpolation to structured parameters. Feature logic impact: all NFT attribute, metadata, and tag updates continue to work exactly as before. The Painless sandbox still runs; the difference is that user data never enters the compiled script.

Test Update Required: No existing tests need to be updated. All existing NFT update tests pass unchanged because the semantic outcome (field values written to ES) is identical. The only change is that injection payloads in `newMetadata` are now stored as literal string values rather than executed as code.

Why This Fix Is Necessary: A script injection that executes with no error and no log entry is the worst class of vulnerability in a compliance system — it produces corrupted data that is indistinguishable from legitimate data.

Silence is worse than explicit failure because a corrupted `kycStatus=approved` field in the compliance index gives no indication to the compliance engineer, the regulatory officer, or the on-call engineer that the value was written by an attacker rather than by the DRWA registry contract.

Finding 2 — REAL FINDING — ES Query Injection — DRWA Revert Delete-by-Query in `elasticProcessor.go` line 432

Classification: - CWE: CWE-943 (Improper Neutralization of Special Elements in Data Query Logic) - CVSS v3.1 Score: **8.1 (High)** — AV:N/AC:H/PR:N/UI:N/S:U/C:N/I:H/A:H - Severity: **High** - Fix Required: Yes - Runtime Impact: Injected `blockHash` in the delete-by-query expands the query scope — wrong documents deleted, or the entire DRWA compliance index wiped on every block revert - Monitoring Impact: Injected delete queries appear as normal revert operations in ES logs — silent data loss with no distinguishing marker

Severity Note: High because `removeFromIndexByBlockHashAndShardID` embeds `blockHash` directly into a raw JSON delete-by-query string via `fmt.Sprintf`. `blockHash` originates from `hex.EncodeToString(header.GetHash())` — bytes received over the WebSocket channel from the connected mx-chain-go node. If the

WebSocket peer is compromised or spoofed, the attacker controls `blockHash` without any access beyond the WebSocket channel. The injected query runs against all six DRWA compliance indices on every block revert: `drwa-denials`, `drwa-identities`, `drwa-holder-compliance`, `drwa-attestations`, `drwa-token-policies`, `drwa-control-events`. A single crafted revert message can permanently delete all KYC/AML records, all attestations, and all denial history from the compliance index.

What the Vulnerable Function Does:

`removeFromIndexByBlockHashAndShardID` constructs a `delete_by_query` request body using `fmt.Sprintf` with `blockHash` and `shardID` embedded as raw string values inside a JSON `term` query. It is called once per DRWA index per block revert — six calls per revert event.

What it does NOT do: It does not validate that `blockHash` contains only hex characters before embedding it. It does not use `json.Marshal` or any escaping mechanism.

Call chain: `RemoveTransactions` → `removedRWRecordsInCaseOfRevert` → `removeFromIndexByBlockHashAndShardID` (×6, once per DRWA index) → `DoQueryRemove` → injected delete-by-query executed by Elasticsearch.

Where Does the Vulnerable Data Come From:

`mx-chain-go` node emits `RevertIndexedBlock` message over WebSocket → `wsindexer.ProcessPayload` deserialises `outport.BlockData` → `dataIndexer.RevertIndexedBlock` called → `elasticProcessor.RemoveTransactions` called → `blockProc.ComputeHeaderHash(header)` computes hash bytes → `hex.EncodeToString(headerHash)` produces hex string → `removeDRWRecordsInCaseOfRevert(shardID, blockHash)` called → `removeFromIndexByBlockHashAndShardID` embeds `blockHash` raw into `fmt.Sprintf` → Elasticsearch executes the delete-by-query.

If the WebSocket peer is a compromised node, `outport.BlockData.HeaderBytes` is attacker-controlled → `ComputeHeaderHash` hashes attacker-controlled bytes → `hex.EncodeToString` of attacker-controlled bytes → `blockHash` is attacker-controlled hex string → injection payload delivered.

Who Uses This Data and Why It Must Be Trusted:

- 1. DevOps (Node Operator):** Monitors DRWA index document counts for consistency after revert events. Breaks if an injected delete-by-query wipes more documents than the reverted block produced. Why watching matters: a sudden drop in `drwa-holder-compliance` document count after a revert is the only signal. Silent failure consequence: operators assume the revert was clean; all KYC/AML records for unrelated blocks are permanently deleted.
- 2. Security (Compliance Engineer):** Relies on the revert path deleting only records from the specific reverted block. Breaks if the query scope is expanded — records from other blocks are deleted. Why watching matters: `drwa-attestations` and `drwa-denials` are append-only audit logs; deletion is irreversible. Silent failure consequence: compliance audit trail has unexplained gaps that cannot be reconstructed from on-chain data alone.
- 3. Compliance (Regulatory Officer):** Must demonstrate complete denial and attestation history for regulatory reporting. Breaks if revert-path injection deletes records from non-reverted blocks. Why watching matters: MiCA requires complete transfer denial records. Silent failure consequence: regulatory filing is missing denial events that actually occurred — potential regulatory violation.

4. **On-Call Engineer:** Receives alerts for unexpected document count drops in DRWA indices. Breaks because the delete-by-query executes as a normal revert operation — no error is logged, no metric is emitted. Why watching matters: the only signal is a document count that is lower than expected after a revert. Silent failure consequence: on-call cannot distinguish a legitimate revert from an injected wipe without replaying all revert events.
-

What an Attacker Can Do:

1. **Full Index Wipe via Crafted Revert:** Attacker controls a WebSocket peer and sends a `RevertIndexedBlock` message with a `HeaderHash` whose hex encoding contains a `match_all` injection.
 - Crafted input: `HeaderHash` bytes chosen so `hex.EncodeToString` produces `aabb"}}`, "should": [{"match_all": {}}], "minimum_should_match": 1, "boost
 - Exact log output: No error — Elasticsearch deletes all documents in the index and returns HTTP 200
 - Consequence: Entire `drwa-holder-compliance` index wiped — all KYC/AML records permanently deleted.
 2. **Cross-Block Record Deletion:** Attacker injects a `range` query that deletes all records with `blockRound` less than a target value — erasing history up to a specific point.
 - Crafted input: `blockHash` injection: `x"}}`, "must": [{"range": {"blockRound": {"lt": 1000000}}}
 - Exact log output: No error — range query executes silently
 - Consequence: All DRWA compliance records before round 1,000,000 are permanently deleted.
 3. **Targeted Attestation Deletion:** Attacker injects a `term` query on `auditor` field to delete all attestation records for a specific auditor address.
 - Crafted input: `blockHash` injection: `x"}}`, "must": [{"term": {"auditor": "erd1targetauditor"}}}
 - Exact log output: No error — targeted delete executes silently
 - Consequence: All attestation records for the targeted auditor are deleted — auditor appears to have never attested anything.
 4. **Repeated Revert Griefing:** Attacker sends repeated `RevertIndexedBlock` messages with the same injected `blockHash`. Each message triggers six delete-by-query calls across all DRWA indices.
 - Crafted input: Same crafted revert message sent 100 times
 - Exact log output: No error per call — 600 total delete-by-query executions
 - Consequence: Entire DRWA compliance index history is wiped across all six indices in seconds.
-

Why This Is Specific to This Feature:

`removeFromIndexByBlockHashAndShardID` is called exclusively for DRWA compliance indices — it is the only delete path that uses `blockHash` as a query parameter embedded via `fmt.Sprintf`. All other delete paths in the indexer use `converters.PrepareHashesForQueryRemove` which builds queries via `json.Marshal`. This function is the highest-risk instance because it runs six times per revert event across the most sensitive compliance data in the system.

The Fix:

BEFORE:

```

func (ei *elasticProcessor) removeFromIndexByBlockHashAndShardID(shardID uint32, index string,
    blockHash string) error {
    ctxWithValue := context.WithValue(context.Background(), request.ContextKey,
        request.ExtendTopicWithShardID(request.RemoveTopic, shardID))

    query := fmt.Sprintf(
        `{"query": {"bool": {"must": [{"term": {"shardID": %d}}, {"term": {"blockHash":
        "%s"}}]}}}`,
        shardID,
        blockHash, // ← raw string – injection point
    )

    return ei.elasticClient.DoQueryRemove(ctxWithValue, index, bytes.NewBuffer([]byte(query)))
}

```

AFTER:

```

func (ei *elasticProcessor) removeFromIndexByBlockHashAndShardID(shardID uint32, index string,
    blockHash string) error {
    ctxWithValue := context.WithValue(context.Background(), request.ContextKey,
        request.ExtendTopicWithShardID(request.RemoveTopic, shardID))

    query := map[string]interface{}{
        "query": map[string]interface{}{
            "bool": map[string]interface{}{
                "must": []interface{}{
                    map[string]interface{}{"term": map[string]interface{}{"shardID": shardID}},
                    map[string]interface{}{"term": map[string]interface{}{"blockHash":
                    blockHash}},
                },
            },
        },
    }
    encoded, err := json.Marshal(query)
    if err != nil {
        return err
    }

    return ei.elasticClient.DoQueryRemove(ctxWithValue, index, bytes.NewBuffer(encoded))
}

```

What each line does: - `map[string]interface{}{...}` — query is built as a typed Go structure; no string interpolation - `json.Marshal(query)` — Go's JSON encoder escapes all special characters in `blockHash` — quotes, backslashes, and control characters are all neutralised - Remove `fmt.Sprintf` — eliminates the injection surface entirely - `encoded, err := json.Marshal(query)` — error is checked; malformed input cannot produce a valid query

Additionally — add a hex-format guard as defence-in-depth before `blockHash` reaches any query builder:

```

func validateBlockHash(blockHash string) error {
    if len(blockHash) == 0 {
        return errors.New("empty block hash")
    }
    for _, c := range blockHash {
        if !((c >= '0' && c <= '9') || (c >= 'a' && c <= 'f')) {
            return fmt.Errorf("invalid character in block hash: %c", c)
        }
    }
}

```

```
    return nil
}
```

Why This Fix Is Safe: No new imports needed beyond `encoding/json` which is already imported. Runtime impact: identical query semantics — only the construction mechanism changes. Feature logic impact: revert path continues to delete exactly the records belonging to the reverted block. All six DRWA index calls are fixed by changing one function.

Test Update Required: No existing tests need to be updated. All existing revert tests pass unchanged because the query semantics are identical for valid hex block hashes. The fix only changes behaviour for crafted non-hex inputs, which had no passing test covering them.

Why This Fix Is Necessary: A delete-by-query that can be scope-expanded by a compromised WebSocket peer is a single-message total data loss vector for the entire DRWA compliance index.

Silence is worse than explicit failure because a wiped `drwa-holder-compliance` index with no error log gives no indication to the compliance engineer, the regulatory officer, or the on-call engineer that the deletion was caused by an injected query rather than a legitimate revert.

Finding 3 — REAL FINDING — ES Query Injection — DRWA Finalize Update-by-Query in `elasticProcessor.go` line 443

Classification: - CWE: CWE-943 (Improper Neutralization of Special Elements in Data Query Logic) - CVSS v3.1 Score: **7.5 (High)** — AV:N/AC:H/PR:N/UI:N/S:U/C:N/I:H/A:H - Severity: **High** - Fix Required: Yes - Runtime Impact: Injected `blockHash` in the finalize update-by-query expands the query scope — `isFinalized=true` is written to DRWA records from unintended blocks, or the entire compliance index is mass-finalized in a single message - Monitoring Impact: Injected update-by-query executes as a normal finalize operation — no error emitted, no distinguishing marker in ES logs

Severity Note: High because `prepareDRWAFinalizedBlockQuery` embeds `blockHash` directly into a raw JSON update-by-query string via `fmt.Sprintf`. The function is called from `FinalizedBlock` which receives `finalizedBlock.HeaderHash` over the WebSocket channel. If the WebSocket peer is compromised, `HeaderHash` is attacker-controlled bytes — `hex.EncodeToString` of attacker-controlled bytes produces an attacker-controlled string that is embedded raw into the Painless update-by-query. The injected query runs against all six DRWA compliance indices on every finalized block. Mass-finalizing unfinalized records is a compliance integrity violation — downstream consumers treat `isFinalized=true` as a guarantee that the block containing the record has been confirmed irreversible on-chain.

What the Vulnerable Function Does:

`prepareDRWAFinalizedBlockQuery` builds an Elasticsearch `update_by_query` payload that sets `isFinalized=true` on all DRWA records matching a given `blockHash` and `shardID`. It uses `fmt.Sprintf` to embed `blockHash` as a raw string inside a JSON `term` query. The result is passed to `UpdateByQuery` which executes it against each of the six DRWA compliance indices.

What it does NOT do: It does not validate that `blockHash` contains only hex characters. It does not use `json.Marshal` or any escaping mechanism. The Painless script source `ctx._source.isFinalized = true` is static and safe — the injection risk is in the query filter, not the script body.

Call chain: `FinalizedBlock` → `hex.EncodeToString(finalizedBlock.HeaderHash)` → `prepareDRWAFinalizedBlockQuery(blockHashHex, shardID)` → `fmt.Sprintf` embeds raw `blockHash` → `UpdateByQuery` (×6, once per DRWA index) → Elasticsearch executes injected update-by-query.

Where Does the Vulnerable Data Come From:

mx-chain-go node emits `FinalizedBlock` message over `WebSocket` → `wsindexer.ProcessPayload` deserialises `outport.FinalizedBlock` → `dataIndexer.FinalizedBlock` called → `elasticProcessor.FinalizeBlock` called → `hex.EncodeToString(finalizedBlock.HeaderHash)` produces hex string → `prepareDRWAFinalizedBlockQuery(blockHashHex, finalizedBlock.ShardID)` called → `fmt.Sprintf` embeds `blockHashHex` raw into query → `UpdateByQuery` executes against all six DRWA indices.

If the `WebSocket` peer is a compromised node, `outport.FinalizedBlock.HeaderHash` is attacker-controlled bytes → `hex.EncodeToString` of attacker-controlled bytes → `blockHashHex` is attacker-controlled → injection payload delivered.

Who Uses This Data and Why It Must Be Trusted:

- 1. DevOps (Node Operator):** Monitors `isFinalized` field transitions in DRWA indices to confirm block finality propagation. Breaks if an injected update-by-query mass-finalizes records from unconfirmed blocks. Why watching matters: a sudden spike in `isFinalized=true` documents across all DRWA indices after a single finalize message is the only signal. Silent failure consequence: operators assume finality propagation is working correctly; unconfirmed records are treated as final by all downstream consumers.
 - 2. Security (Compliance Engineer):** Relies on `isFinalized=true` as a guarantee that the block containing the record has been confirmed irreversible on-chain. Breaks if an injected query sets `isFinalized=true` on records from blocks that have not yet been finalized. Why watching matters: downstream compliance consumers use `isFinalized` to gate regulatory reporting — only finalized records are included in reports. Silent failure consequence: unconfirmed compliance records are included in regulatory reports as if they were final.
 - 3. Compliance (Regulatory Officer):** Must demonstrate that reported compliance events correspond to irreversible on-chain state. Breaks if `isFinalized=true` is set on records from reverted or unconfirmed blocks. Why watching matters: a compliance record marked finalized that corresponds to a reverted block is a false regulatory event. Silent failure consequence: regulatory filing includes events that never actually occurred on the canonical chain.
 - 4. On-Call Engineer:** Receives alerts for unexpected `isFinalized` transitions. Breaks because the injected update-by-query executes as a normal finalize operation — no error is logged, no metric is emitted. Why watching matters: the only signal is an `isFinalized` count that is higher than the number of finalized blocks. Silent failure consequence: on-call cannot distinguish a legitimate finalize from an injected mass-finalize without replaying all finalize events.
-

What an Attacker Can Do:

- 1. Mass Finalize via Crafted FinalizedBlock:** Attacker controls a `WebSocket` peer and sends a `FinalizedBlock` message with a `HeaderHash` whose hex encoding contains a `match_all` injection.
 - Crafted input: `HeaderHash` bytes chosen so `hex.EncodeToString` produces `aabb}}}}, "should": [{"match_all": {}}], "minimum_should_match": 1, "boost`
 - Exact log output: No error — Elasticsearch sets `isFinalized=true` on every document in all six DRWA indices and returns HTTP 200
 - Consequence: All DRWA compliance records — including records from unconfirmed and reverted blocks — are permanently marked as finalized.

2. **Cross-Shard Finalize Injection:** Attacker injects a query that finalizes records from a different shard than the one in the `FinalizedBlock` message.
 - Crafted input: `blockHash` injection: `x"}}`}, "must": [{"term": {"shardID": 0}}}
 - Exact log output: No error — records from shard 0 are finalized regardless of the message's shard ID
 - Consequence: Records from a different shard are incorrectly marked as finalized.
3. **Targeted Identity Finalization:** Attacker injects a `term` query on `subject` field to finalize all identity records for a specific address — including records from blocks that have not yet been finalized.
 - Crafted input: `blockHash` injection: `x"}}`}, "must": [{"term": {"subject": "erd1targetaddress"}}}
 - Exact log output: No error — all identity records for the target address are marked finalized
 - Consequence: Unconfirmed identity registrations for the target address appear final in all compliance dashboards.
4. **Repeated Finalize Griefing:** Attacker sends repeated `FinalizedBlock` messages with the same injected `blockHash`. Each message triggers six update-by-query calls across all DRWA indices.
 - Crafted input: Same crafted finalize message sent 100 times
 - Exact log output: No error per call — 600 total update-by-query executions, all idempotent after the first
 - Consequence: All DRWA compliance records are permanently marked finalized after the first message; subsequent messages are no-ops but consume ES resources.

Why This Is Specific to This Feature:

`prepareDRWAFinalizedBlockQuery` is the only update-by-query builder in the entire indexer that embeds a string parameter via `fmt.Sprintf`. All other update-by-query calls in the codebase either use only numeric parameters or use `json.Marshal`. This function is called exclusively for DRWA compliance indices — it is the finality propagation mechanism for the most sensitive compliance data in the system. Corrupting `isFinalized` is uniquely dangerous because it is a one-way transition: once set to `true`, downstream consumers treat the record as permanently confirmed.

The Fix:

BEFORE:

```
func prepareDRWAFinalizedBlockQuery(blockHash string, shardID uint32) *bytes.Buffer {
    query := fmt.Sprintf(`{
    "script": {
        "source": "ctx._source.isFinalized = true",
        "lang": "painless"
    },
    "query": {
        "bool": {
            "must": [
                { "term": { "blockHash": "%s" } },
                { "term": { "shardID": %d } }
            ]
        }
    }
}`)
    return bytes.NewBuffer(query)
}
```

```
    return bytes.NewBuffer([]byte(query))
}
```

AFTER:

```
func prepareDRWAFinalizedBlockQuery(blockHash string, shardID uint32) (*bytes.Buffer, error) {
    query := map[string]interface{}{
        "script": map[string]interface{}{
            "source": "ctx._source.isFinalized = true",
            "lang":   "painless",
        },
        "query": map[string]interface{}{
            "bool": map[string]interface{}{
                "must": []interface{}{
                    map[string]interface{}{"term": map[string]interface{}{"blockHash":
                        blockHash}},
                    map[string]interface{}{"term": map[string]interface{}{"shardID": shardID}},
                },
            },
        },
    }
    encoded, err := json.Marshal(query)
    if err != nil {
        return nil, err
    }
    return bytes.NewBuffer(encoded), nil
}
```

Update the caller in `FinalizedBlock` to handle the error return:

```
// FIXED - FinalizedBlock in elasticProcessor.go
query, err := prepareDRWAFinalizedBlockQuery(blockHashHex, finalizedBlock.ShardID)
if err != nil {
    return err
}
```

What each line does: - `map[string]interface{}{...}` — query is built as a typed Go structure; no string interpolation - `json.Marshal(query)` — Go's JSON encoder escapes all special characters in `blockHash` - Return `(*bytes.Buffer, error)` — error is now surfaced to the caller instead of silently producing a malformed query - Remove `fmt.Sprintf` — eliminates the injection surface entirely - Painless `source` string `ctx._source.isFinalized = true` is unchanged — it was always static and safe

Why This Fix Is Safe: No new imports needed beyond `encoding/json` which is already imported. Runtime impact: identical query semantics for valid hex block hashes. Feature logic impact: finality propagation continues to work exactly as before. The signature change from `*bytes.Buffer` to `(*bytes.Buffer, error)` is a one-line update at the single call site in `FinalizedBlock`.

Test Update Required: No existing tests need to be updated. All existing finalize tests pass unchanged because the query semantics are identical for valid hex block hashes. The fix only changes behaviour for crafted non-hex inputs, which had no passing test covering them.

Why This Fix Is Necessary: A finality flag that can be mass-set by a single crafted WebSocket message is a compliance integrity violation — it makes `isFinalized=true` meaningless as a guarantee of on-chain confirmation.

Silence is worse than explicit failure because a mass-finalized compliance index with no error log gives no indication to the compliance engineer, the regulatory officer, or the on-call engineer that the `isFinalized` transitions were caused by an injected query rather than legitimate block finality events.

Finding 4 — REAL FINDING — Missing Elasticsearch Authentication in `prefs.toml` line 21

Classification: - CWE: CWE-306 (Missing Authentication for Critical Function) - CVSS v3.1 Score: **9.1 (High)** — AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N - Severity: **High** - Fix Required: Yes - Runtime Impact: Entire Elasticsearch cluster is accessible without credentials — all indexed blockchain data, all DRWA compliance records, all KYC/AML status, all attestations can be read, modified, or deleted by any network-reachable host - Monitoring Impact: No audit trail exists — all access to indexed data is invisible; there is no way to determine whether data was accessed or modified by an attacker

Severity Note: High because the Elasticsearch cluster stores every indexed blockchain event — transactions, accounts, DRWA denial records, holder compliance mirrors, identity registrations, attestations, and token policies. Empty `username` and `password` fields in `prefs.toml` mean the indexer connects with zero authentication. The Elasticsearch Go client sends requests with no `Authorization` header. Any host with network access to port 9200 can read, write, or delete the entire dataset. DRWA compliance data is subject to financial regulatory requirements — unauthenticated access is a direct regulatory violation independent of whether an attacker exploits it.

What the Vulnerable Function Does:

`prefs.toml` defines the Elasticsearch cluster connection parameters. The `username` and `password` fields are empty strings. `loadClusterConfig` in `main.go` reads this file and passes the values directly into `ArgsIndexerFactory.createElasticClient` in `indexerFactory.go` constructs an `elasticsearch.Config` with `Username: args.UserName` and `Password: args.Password` — both empty strings. The Elasticsearch Go client omits the `Authorization` header when credentials are empty, resulting in unauthenticated HTTP requests to the cluster.

What it does NOT do: It does not validate that credentials are non-empty before constructing the client. It does not fail fast at startup if credentials are missing. It does not enforce TLS — the configured URL uses `http://` not `https://`.

Call chain: `main` → `loadClusterConfig` → `CreateWsIndexer` → `createDataIndexer` → `createElasticClient` → `elasticsearch.NewClient(cfg)` → all subsequent ES requests sent without `Authorization` header.

Where Does the Vulnerable Data Come From:

`prefs.toml` is an operator-supplied configuration file read at startup. The `username` and `password` fields are empty strings in the committed template. The vulnerability is not that the file is committed with real credentials — it is that the application accepts empty credentials and starts successfully, connecting to Elasticsearch without authentication. Any operator who deploys the indexer using the default configuration exposes the entire cluster.

Who Uses This Data and Why It Must Be Trusted:

1. **DevOps (Node Operator):** Relies on Elasticsearch access being restricted to the indexer service. Breaks if any host on the network can connect directly to port 9200 without credentials. Why watching matters: an open Elasticsearch port is detectable by any network scanner. Silent failure consequence:

operators assume the cluster is protected by network segmentation alone — a single misconfigured firewall rule exposes all data.

2. **Security (Compliance Engineer):** Relies on access controls to ensure only authorised services can read or write DRWA compliance records. Breaks if unauthenticated access is possible — any process on the network can read KYC/AML status, attestation records, and denial history. Why watching matters: DRWA compliance data includes personally identifiable information (subject addresses, jurisdiction codes, KYC/AML status). Silent failure consequence: PII is exposed to any network-reachable host with no audit trail.
 3. **Compliance (Regulatory Officer):** Must demonstrate that access to compliance data is restricted and audited. Breaks if no authentication is enforced — there is no way to produce an access audit log. Why watching matters: MiCA Article 30 and equivalent regulations require access controls and audit trails for compliance data. Silent failure consequence: regulatory audit finds no access controls on compliance data — potential regulatory sanction.
 4. **On-Call Engineer:** Relies on Elasticsearch authentication to prevent unauthorised data modification. Breaks if an attacker modifies indexed data — the on-call engineer cannot distinguish legitimate indexed data from attacker-injected data. Why watching matters: without authentication, there is no way to determine who wrote a given document. Silent failure consequence: on-call cannot determine whether a data anomaly was caused by an indexer bug or an external attacker.
-

What an Attacker Can Do:

1. **Full Data Exfiltration:** Attacker with network access to port 9200 dumps all DRWA compliance indices without credentials.
 - Crafted input: `curl http://localhost:9200/drwa-holder-compliance/_search?size=10000`
 - Exact log output: No log — Elasticsearch returns HTTP 200 with all documents
 - Consequence: All KYC/AML status, jurisdiction codes, investor classes, and holder addresses are exfiltrated with no audit trail.
2. **Compliance Record Injection:** Attacker injects a fake holder compliance record marking a non-compliant address as KYC/AML approved.
 - Crafted input: `curl -X POST http://localhost:9200/drwa-holder-compliance/_doc/fake-id -d '{"holder":"erd1attacker","kycStatus":"approved","amlStatus":"approved"}'`
 - Exact log output: No log — Elasticsearch returns HTTP 201 Created
 - Consequence: Attacker's address appears KYC/AML approved in the compliance index; regulated transfers proceed for a non-compliant holder.
3. **Compliance Index Deletion:** Attacker deletes the entire `drwa-holder-compliance` index.
 - Crafted input: `curl -X DELETE http://localhost:9200/drwa-holder-compliance`
 - Exact log output: No log — Elasticsearch returns HTTP 200 with `{"acknowledged":true}`
 - Consequence: All holder compliance records are permanently deleted; compliance dashboard shows no holders; all regulated transfers are blocked until the index is rebuilt.
4. **Transaction History Tampering:** Attacker modifies indexed transaction records to alter sender, receiver, or value fields.
 - Crafted input: `curl -X POST http://localhost:9200/transactions/_update/tx-hash -d '{"doc":{"value":"0","status":"fail"}}'`
 - Exact log output: No log — Elasticsearch returns HTTP 200

- Consequence: Indexed transaction history is corrupted; downstream consumers (block explorers, compliance tools) show incorrect transaction data.

Why This Is Specific to This Feature:

The indexer is the sole writer of DRWA compliance data to Elasticsearch. Without authentication, the trust boundary between the indexer and the cluster does not exist — any process can write compliance data directly, bypassing all DRWA event validation, canonical event allowlist checks, and contract address verification. This makes authentication not just a security control but a prerequisite for the correctness of the entire DRWA compliance pipeline.

The Fix:

BEFORE:

```
[config.elastic-cluster]
  url      = "http://localhost:9200"
  username = ""
  password = ""
  bulk-request-max-size-in-bytes = 4194304
```

AFTER — Step 1, enforce credentials at startup in `factory/wsIndexerFactory.go`:

```
// FIXED - fail fast if credentials are missing
func createDataIndexer(cfg config.Config, clusterCfg config.ClusterConfig, ...)
(wsindexer.DataIndexer, error) {
  if clusterCfg.Config.ElasticCluster.UserName == "" ||
    clusterCfg.Config.ElasticCluster.Password == "" {
    return nil, errors.New("elasticsearch credentials must not be empty - set ES_USERNAME
    and ES_PASSWORD")
  }
  // ... rest of function unchanged
}
```

AFTER — Step 2, load credentials from environment variables in `cmd/elasticindexer/main.go`:

```
// FIXED - override config with environment variables
if u := os.Getenv("ES_USERNAME"); u != "" {
  clusterCfg.Config.ElasticCluster.UserName = u
}
if p := os.Getenv("ES_PASSWORD"); p != "" {
  clusterCfg.Config.ElasticCluster.Password = p
}
```

AFTER — Step 3, switch to HTTPS in `prefs.toml`:

```
[config.elastic-cluster]
  url      = "https://localhost:9200"
  username = ""
  password = ""
  bulk-request-max-size-in-bytes = 4194304
```

AFTER — Step 4, enable xpack security in Elasticsearch:

```
# elasticsearch.yml
xpack.security.enabled: true
xpack.security.http.ssl.enabled: true
xpack.security.audit.enabled: true
```

What each line does: - `errors.New("elasticsearch credentials must not be empty")` — application refuses to start without credentials; no silent unauthenticated deployment possible - `os.Getenv("ES_USERNAME")` — credentials come from environment, never from a committed file - `url = "https://..."` — enforces TLS; credentials are not sent in plaintext - `xpack.security.enabled: true` — enables authentication, authorisation, and audit logging on the cluster - `xpack.security.audit.enabled: true` — every read and write is logged with the authenticated user identity

Why This Fix Is Safe: No runtime behaviour changes for correctly configured deployments. The fail-fast check at startup prevents silent unauthenticated deployments. Environment variable override is additive — existing deployments that already set credentials via environment continue to work unchanged.

Test Update Required: No existing tests need to be updated. Integration tests that use a local Elasticsearch instance should set `ES_USERNAME` and `ES_PASSWORD` in the test environment. Unit tests do not connect to Elasticsearch and are unaffected.

Why This Fix Is Necessary: An unauthenticated Elasticsearch cluster storing DRWA compliance data is a direct violation of the access control requirements of MiCA, GDPR, and equivalent financial data regulations.

Silence is worse than explicit failure because an indexer that starts successfully with empty credentials gives no indication to the operator, the compliance engineer, or the regulatory officer that the entire compliance dataset is accessible to any network-reachable host.

Finding 5 — REAL FINDING — CORS AllowAllOrigins with Authorization Header in

`webServer.go` line 61

Classification: - CWE: CWE-942 (Permissive Cross-domain Policy with Untrusted Domains) - CVSS v3.1 Score: **7.5 (High)** — AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N - Severity: **High** - Fix Required: Yes - Runtime Impact: Any malicious website can make authenticated cross-origin requests to the indexer API from a victim's browser and read all metrics and blockchain data responses - Monitoring Impact: Legitimate cross-origin requests cannot be distinguished from malicious ones — no per-origin access log exists

Severity Note: High because `AllowAllOrigins = true` combined with `AddAllowHeaders("Authorization")` means any website on the internet can instruct a victim's browser to send requests — including bearer tokens — to the indexer API and read the responses. The current endpoints (`/status/metrics`, `/status/prometheus-metrics`) expose Elasticsearch cluster health, indexing throughput, error rates, and block processing latency. This data is sufficient for an attacker to map the infrastructure, identify degraded states, and time attacks. The Authorization header allowance means any future authenticated endpoint is immediately exploitable with zero additional changes required.

What the Vulnerable Function Does:

`StartHttpServer` in `webServer.go` initialises the Gin engine and attaches a CORS middleware. It sets `AllowAllOrigins = true` with no origin whitelist, no method restriction beyond Gin defaults, and explicitly adds the `Authorization` header to the allowed list. Every route registered on the engine — current and future — inherits this policy.

What it does NOT do: It does not restrict allowed origins to a known set. It does not restrict allowed methods to read-only. It does not remove the `Authorization` header from the allowed list for unauthenticated endpoints.

Call chain: `main` → `startIndexer` → `factory.CreateWebServer` → `webServer.StartHttpServer` → `cors.New(cfg)` applied to Gin engine → all routes inherit unrestricted CORS policy.

Where Does the Vulnerable Data Come From:

`api.toml` sets `rest-api-interface = ":8080"` — the server binds on all interfaces. `prefs.toml` has no CORS configuration. The CORS policy is hardcoded in `webServer.go` with no configuration hook. The data exposed by the metrics endpoints originates from the internal `StatusMetrics` counter set — Elasticsearch request counts, durations, error rates, and topic-level indexing statistics.

Who Uses This Data and Why It Must Be Trusted:

1. **DevOps (Node Operator):** Uses `/status/prometheus-metrics` to feed Grafana dashboards. Breaks if an attacker reads these metrics from a victim's browser — infrastructure topology, error rates, and indexing lag are exposed. Why watching matters: metrics reveal whether the indexer is degraded, which topics are failing, and how many ES requests are being retried. Silent failure consequence: attacker uses metrics to time an attack during a degraded state when the indexer is already retrying ES requests.
 2. **Security (Infrastructure Engineer):** Relies on the metrics endpoint being accessible only to authorised monitoring systems. Breaks if any website can read metrics via a victim's browser — internal infrastructure details are exposed to arbitrary third parties. Why watching matters: Prometheus metrics include ES cluster URL fragments, topic names, and shard IDs. Silent failure consequence: attacker maps the internal infrastructure from metrics data without ever connecting directly to the indexer.
 3. **Compliance (Regulatory Officer):** Relies on access to compliance infrastructure being restricted. Breaks if metrics about DRWA indexing throughput and error rates are readable by any website. Why watching matters: metrics reveal whether DRWA compliance indexing is functioning — an attacker who knows the DRWA indexer is degraded can time a compliance bypass attempt. Silent failure consequence: attacker knows exactly when the compliance pipeline is most vulnerable.
 4. **On-Call Engineer:** Relies on the metrics endpoint being a trusted source of infrastructure state. Breaks if an attacker can read metrics from a victim's browser and use them to craft targeted attacks. Why watching matters: metrics include per-topic error counts that reveal which parts of the indexer are failing. Silent failure consequence: on-call cannot determine whether a metrics anomaly was caused by a legitimate infrastructure issue or by an attacker probing the system.
-

What an Attacker Can Do:

1. **Metrics Exfiltration via Victim Browser:** Attacker hosts a malicious website that silently fetches indexer metrics from a victim's browser and exfiltrates them.
 - Crafted input: `fetch('http://indexer:8080/status/metrics', {credentials: 'include', headers: {'Authorization': 'Bearer stolen-token'}})`
 - Exact log output: No log — Gin returns HTTP 200 with full metrics JSON; CORS headers allow the browser to read the response

- Consequence: Attacker receives ES cluster health, indexing throughput, error rates, and topic-level statistics from the victim's internal network.

2. Infrastructure Mapping: Attacker reads Prometheus metrics to extract ES cluster URL, topic names, shard IDs, and indexing lag.

- Crafted input: `fetch('http://indexer:8080/status/prometheus-metrics')`
- Exact log output: No log — Gin returns HTTP 200 with Prometheus-format metrics
- Consequence: Attacker maps internal infrastructure topology without ever connecting directly to the indexer or Elasticsearch.

3. Attack Timing via Degradation Detection: Attacker polls metrics every 30 seconds to detect when the indexer is in a degraded state (high retry count, elevated error rate) and times a compliance bypass attempt during the degraded window.

- Crafted input: Repeated `fetch` calls to `/status/prometheus-metrics` from a background script on a malicious website
- Exact log output: No log per call — each returns HTTP 200
- Consequence: Attacker knows exactly when the DRWA compliance pipeline is most vulnerable to a timing attack.

4. Future Authenticated Endpoint Exploitation: When a future authenticated endpoint is added to the indexer API, the existing CORS policy with `Authorization` header allowance makes it immediately exploitable from any website without any additional configuration change.

- Crafted input: `fetch('http://indexer:8080/admin/reindex', {method: 'POST', headers: {'Authorization': 'Bearer victim-token'}})`
- Exact log output: No log — CORS policy allows the request; browser sends the victim's token
- Consequence: Any future authenticated endpoint is exploitable from any website the moment it is deployed.

Why This Is Specific to This Feature:

The CORS policy is set once in `StartHttpServer` and applies to every route on the engine — current and future. The explicit addition of `Authorization` to the allowed headers is the most dangerous aspect: it is not required by any current endpoint (both metrics endpoints are unauthenticated) but it pre-authorises credential forwarding for all future endpoints. This is a security debt that compounds with every new endpoint added to the API.

The Fix:

BEFORE:

```
engine = gin.Default()
cfg := cors.DefaultConfig()
cfg.AllowAllOrigins = true
cfg.AddAllowHeaders("Authorization")
engine.Use(cors.New(cfg))
```

AFTER:

```

engine = gin.Default()
cfg := cors.DefaultConfig()

// Load allowed origins from environment – default to empty (no cross-origin access)
allowedOrigins := os.Getenv("ALLOWED_ORIGINS")
if allowedOrigins != "" {
    cfg.AllowOrigins = strings.Split(allowedOrigins, ",")
} else {
    cfg.AllowOrigins = []string{} // no cross-origin access by default
}
cfg.AllowMethods = []string{"GET"} // metrics endpoints are read-only
cfg.AllowHeaders = []string{"Content-Type"} // remove Authorization – not needed by any current endpoint
engine.Use(cors.New(cfg))

```

What each line does: - `cfg.AllowOrigins = []string{} — no cross-origin access by default; operator must explicitly configure trusted origins` - `os.Getenv("ALLOWED_ORIGINS") — origins are configurable without code changes; set to monitoring dashboard URLs in production` - `cfg.AllowMethods = []string{"GET"} — restricts cross-origin requests to read-only; POST/DELETE cannot be made cross-origin` - Remove `cfg.AddAllowHeaders("Authorization") — eliminates pre-authorisation of credential forwarding for future endpoints` - `cfg.AllowHeaders = []string{"Content-Type"} — only the minimum required header is allowed`

Why This Fix Is Safe: No runtime behaviour changes for server-side consumers (Prometheus scrapers, monitoring agents) — they do not send `origin` headers and are unaffected by CORS policy. Only browser-initiated cross-origin requests are affected. Operators who need cross-origin access set `ALLOWED_ORIGINS` to their dashboard domain.

Test Update Required: No existing tests need to be updated. CORS middleware is not tested in the current test suite. All existing API tests make direct requests without `origin` headers and are unaffected.

Why This Fix Is Necessary: A CORS policy that allows all origins with Authorization header forwarding is a standing invitation for any website to exfiltrate infrastructure metrics and pre-authorises credential theft for every future authenticated endpoint.

Silence is worse than explicit failure because a metrics response delivered to a malicious website via a victim's browser produces no log entry, no error, and no alert — the exfiltration is completely invisible to the operator, the security engineer, and the on-call engineer.

Finding 6 — REAL FINDING — ES Query Injection via On-Chain Token Identifier in `query.go` line 48

Classification: - CWE: CWE-943 (Improper Neutralization of Special Elements in Data Query Logic) - CVSS v3.1 Score: **6.5 (Medium)** — AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N - Severity: **Medium** - Fix Required: Yes - Runtime Impact: Injected token identifier or address expands the query scope — all documents in the index are returned instead of only the intended token's records - Monitoring Impact: Injected queries are indistinguishable from legitimate queries in ES logs

Severity Note: Medium because `queryGetLastTxForToken` and `queryGetLastOperationForAddress` embed `identifier` and `addr` directly into raw JSON via `fmt.Sprintf` with no escaping. These values originate from account records scrolled out of Elasticsearch, which themselves originate from on-chain state. A malicious actor who registers a token with a crafted identifier can inject arbitrary Elasticsearch query syntax. This is in the `accounts-balance-checker` tool — not the main indexer — but the tool has direct read access to the same Elasticsearch cluster that stores all DRWA compliance data.

What the Vulnerable Function Does:

`queryGetLastTxForToken` builds a bool/must query to find the last transaction for a given token and sender. `queryGetLastOperationForAddress` builds a bool/should query to find the last operation for an address. Both use `fmt.Sprintf` to embed raw string values into the JSON query body.

Call chain: `CheckESDTBalances` → `handlerFuncScrollAccountESDT` → `checkBalance` → `getLastTimeWhenBalanceWasChanged` → `queryGetLastTxForToken(identifier, addr)` → injected query sent to ES.

Where Does the Vulnerable Data Come From:

On-chain token registration → token identifier stored in MultiversX trie → indexer writes identifier to `accounts` index → balance checker scrolls `accounts` index → `identifier` field read from ES response → passed to `queryGetLastTxForToken` → embedded raw into `fmt.Sprintf` → injected query sent to ES.

Who Uses This Data and Why It Must Be Trusted:

1. **DevOps (Node Operator):** Uses the balance checker to verify indexed balances match on-chain state. Breaks if an injected query returns all documents — the checker processes thousands of unintended records. Silent failure consequence: balance checker reports false mismatches for every account in the index.
 2. **Security (Compliance Engineer):** Relies on the balance checker querying only the intended token's records. Breaks if injection expands the scope — unintended DRWA compliance records are processed. Silent failure consequence: compliance records for unrelated tokens are incorrectly flagged as balance mismatches.
 3. **Compliance (Regulatory Officer):** Relies on balance checker output for reconciliation reports. Breaks if injected queries return unintended data — reconciliation report contains records from unrelated tokens. Silent failure consequence: regulatory reconciliation report is corrupted with unrelated data.
 4. **On-Call Engineer:** Uses the balance checker to diagnose indexing issues. Breaks if an injected query causes the checker to process the entire index — tool runs indefinitely and consumes all ES resources. Silent failure consequence: on-call cannot use the balance checker during an incident because it is stuck processing injected results.
-

What an Attacker Can Do:

1. **Full Index Scan:** Attacker registers a token with identifier `TOKEN-a1b2c3`, `"operator": "OR"}}`, `{"match_all": {}}` — balance checker query returns all documents.
 - Exact log output: No error — ES returns HTTP 200 with all documents
 - Consequence: Balance checker processes entire transactions index — all account data exfiltrated.
2. **Targeted Data Extraction:** Attacker crafts identifier to inject a `term` query on a specific address field — extracts all transactions for a target address.
 - Crafted input: `TOKEN-x`, `"operator": "OR"}}`, `{"term": {"sender": "erd1target`
 - Consequence: All transactions for the target address are returned to the balance checker.

3. **Resource Exhaustion:** Attacker crafts identifier to inject a `match_all` query — balance checker processes millions of documents, exhausting ES heap and causing cluster instability.
 - Consequence: ES cluster becomes unresponsive; all indexing operations fail.
4. **Repair Path Exploitation:** If `--repair` flag is set, the balance checker writes back to ES. An injected query that returns unintended documents causes the repair logic to overwrite balances for unintended accounts.
 - Consequence: Correct account balances are overwritten with values from unrelated accounts.

Why This Is Specific to This Feature:

`queryGetLastTxForToken` and `queryGetLastOperationForAddress` are the only two query builders in the balance checker that embed string parameters via `fmt.Sprintf`. All other queries in the tool use `json.Marshal` or structured query objects. The repair path (`fixWrongBalance`, `deleteExtraBalance`) makes this uniquely dangerous — injection on a tool with write access can corrupt the index it is meant to repair.

The Fix:

BEFORE:

```
func queryGetLastTxForToken(identifier, addr string) *bytes.Buffer {
    queryBytes := fmt.Sprintf(`{
        "query": {"bool": {"must": [
            {"match": {"tokens": {"query": "%s", "operator": "AND"}}},
            {"match": {"sender": {"query": "%s", "operator": "AND"}}}
        ]}},
        "sort": [{"timestamp": {"order": "desc"}}]
    }`, identifier, addr)
    return bytes.NewBuffer([]byte(queryBytes))
}
```

AFTER:

```
func queryGetLastTxForToken(identifier, addr string) (*bytes.Buffer, error) {
    query := map[string]interface{}{
        "query": map[string]interface{}{
            "bool": map[string]interface{}{
                "must": []interface{}{
                    map[string]interface{}{"match": map[string]interface{}{
                        "tokens": map[string]interface{}{"query": identifier, "operator": "AND"}
                    },
                    map[string]interface{}{"match": map[string]interface{}{
                        "sender": map[string]interface{}{"query": addr, "operator": "AND"},
                    },
                },
            },
            "sort": []interface{}{map[string]interface{}{"timestamp": map[string]interface{}{
                "order": "desc"}},
            },
        },
    }
    encoded, err := json.Marshal(query)
    if err != nil {
        return nil, err
    }
}
```

```
}  
    return bytes.NewBuffer(encoded), nil  
}
```

Apply the same fix to `queryGetLastOperationForAddress` — replace `fmt.Sprintf` with `json.Marshal` on a structured map.

Why This Fix Is Safe: No semantic change for valid inputs. `json.Marshal` escapes all special characters in `identifier` and `addr` — injection is structurally impossible regardless of input content.

Test Update Required: No existing tests need to be updated. The fix only changes behaviour for crafted injection inputs which had no passing test.

Why This Fix Is Necessary: A query builder that accepts on-chain values without escaping gives any token creator the ability to read arbitrary data from the Elasticsearch cluster via the balance checker tool.

Silence is worse than explicit failure because an injected `match_all` query returns HTTP 200 with all documents — the balance checker processes the results as if they were legitimate, with no indication that the query scope was expanded by an attacker.

Finding 7 — REAL FINDING — No Contract Address Verification for DRWA Events in `drwaEventsProcessor.go` line 88

Classification: - CWE: CWE-345 (Insufficient Verification of Data Authenticity) - CVSS v3.1 Score: **6.5 (Medium)** — AV:N/AC:L/PR:L/UI:N/S:U/C:N/I:H/A:N - Severity: **Medium** - Fix Required: Yes - Runtime Impact: Any smart contract on MultiversX can emit a `drwaHolderCompliance` or `drwaIdentityRegistered` event and the indexer will process it as a legitimate compliance event — spoofed KYC/AML records are written to the compliance index - Monitoring Impact: Spoofed compliance records are indistinguishable from legitimate ones in the index — no field indicates the emitting contract address

Severity Note: Medium because `drwaCanonicalEventsMap` checks only the event identifier, not the emitting contract address. Any contract can emit an event with identifier `drwaHolderCompliance` and the indexer will process it. The data origin is ON-CHAIN but from an UNTRUSTED contract — the indexer trusts the event identifier but not the event source. For a compliance system handling KYC/AML data, the source of the event is as important as its identifier. An attacker needs only the ability to deploy a contract and emit an event — no special privilege required.

What the Vulnerable Function Does:

`processEvent` in `drwaEventsProcessor.go` checks if the event identifier is in `drwaCanonicalEventsMap`. If yes, it processes the event and writes records to the DRWA compliance indices. It never checks `args.logAddress` — the address of the contract that emitted the event.

What it does NOT do: It does not verify that the event was emitted by the authorised DRWA registry contract. `args.logAddress` is available in `argsProcessEvent` but is never used by `drwaEventsProcessor`.

Call chain: `ExtractDataFromLogs` → `processEvents` → `processEvent` → `drwaEventsProcessor.processEvent` → checks `drwaCanonicalEventsMap[identifier]` → processes event regardless of emitting contract → writes to DRWA indices.

Where Does the Vulnerable Data Come From:

Any smart contract on MultiversX → emits event with identifier `drwaHolderCompliance` → `mx-chain-go` node includes event in `OutputBlock.TransactionPool.Logs` → `WebSocket` delivers to indexer → `ExtractDataFromLogs` processes all logs → `drwaEventsProcessor.processEvent` checks only the identifier → event is processed as legitimate → spoofed compliance record written to `drwa-holder-compliance` index.

Who Uses This Data and Why It Must Be Trusted:

- 1. DevOps (Node Operator):** Monitors DRWA index document counts to track compliance activity. Breaks if spoofed events inflate the counts — document count no longer reflects legitimate compliance events. Why watching matters: a sudden spike in `drwa-holder-compliance` documents with no corresponding increase in legitimate DRWA contract activity is the only signal. Silent failure consequence: operators assume the spike is legitimate; spoofed records are never investigated.
 - 2. Security (Compliance Engineer):** Relies on the compliance index containing only records from the authorised DRWA registry contract. Breaks if any contract can write to the index — compliance dashboards show spoofed KYC/AML approvals. Why watching matters: a holder marked `kycStatus=approved` by a spoofed event can transfer regulated tokens. Silent failure consequence: regulated transfers proceed for non-compliant holders; compliance gate is bypassed.
 - 3. Compliance (Regulatory Officer):** Must demonstrate that all compliance records in the index originate from the authorised registry. Breaks if spoofed events are processed — regulatory audit finds records from unauthorised contracts. Why watching matters: MiCA requires that compliance data originates from authorised sources. Silent failure consequence: regulatory audit finds compliance records with no corresponding authorised registry event — potential regulatory violation.
 - 4. On-Call Engineer:** Investigates compliance record anomalies. Breaks because spoofed records have no distinguishing field — the emitting contract address is not stored in the indexed document. Why watching matters: the only way to detect a spoofed record is to replay the block and check the emitting contract address. Silent failure consequence: on-call cannot determine whether a compliance record is legitimate without replaying the entire blockchain.
-

What an Attacker Can Do:

- 1. Spoofed KYC/AML Approval:** Attacker deploys a contract that emits `drwaHolderCompliance` with `kycStatus=approved`, `amlStatus=approved` for the attacker's address.
 - Crafted input: Contract emits event `drwaHolderCompliance` with topics `[TOKEN-abc, erd1attacker, 0x01, approved, approved, accredited, US, ...]`
 - Exact log output: No error — indexer processes the event and writes to `drwa-holder-compliance`
 - Consequence: Attacker's address appears KYC/AML approved in the compliance index; regulated transfers proceed.
- 2. Spoofed Identity Registration:** Attacker deploys a contract that emits `drwaIdentityRegistered` for a target address with a fake jurisdiction code.
 - Crafted input: Contract emits `drwaIdentityRegistered` with topics `[erd1target, XX, company]`
 - Exact log output: No error — indexer writes to `drwa-identities`
 - Consequence: Target address appears registered in a fake jurisdiction; compliance checks use the spoofed data.

3. **Spoofed Attestation:** Attacker deploys a contract that emits `drwaAttestationRecorded` claiming an auditor approved a subject that was never actually attested.

- Crafted input: Contract emits `drwaAttestationRecorded` with topics `[TOKEN-abc, erd1subject, erd1fakeauditor, kyc, 0x01, 12345]`
- Exact log output: No error — indexer writes to `drwa-attestations`
- Consequence: Fake attestation appears in the compliance index; downstream consumers trust it as legitimate.

4. **Compliance Index Pollution:** Attacker deploys multiple contracts that emit thousands of spoofed DRWA events — compliance index is flooded with fake records.

- Crafted input: 10,000 spoofed `drwaHolderCompliance` events from 100 different contracts
- Exact log output: No error — all events are processed and indexed
- Consequence: Compliance index contains more spoofed records than legitimate ones; compliance dashboards are unusable.

Why This Is Specific to This Feature:

The DRWA event processing pipeline is the only part of the indexer that processes events based solely on their identifier without verifying the emitting contract address. All other event processors in the codebase either process events from known system contracts or do not make trust decisions based on event content. DRWA compliance events are unique because they carry regulatory significance — a spoofed `kycStatus=approved` event has real-world consequences for regulated token transfers.

The Fix:

BEFORE:

```
func (dep *drwaEventsProcessor) processEvent(args *argsProcessEvent) argOutputProcessEvent {
    identifier := string(args.event.GetIdentifier())
    if _, ok := drwaCanonicalEventsMap[strings.ToLower(identifier)]; !ok {
        return argOutputProcessEvent{}
    }
    // processes event regardless of emitting contract address
}
```

AFTER:

```
type drwaEventsProcessor struct {
    drwaRegistryAddress string // bech32 address of authorised DRWA registry
}

func newDRWAEventsProcessor(registryAddress string) *drwaEventsProcessor {
    return &drwaEventsProcessor{drwaRegistryAddress: registryAddress}
}

func (dep *drwaEventsProcessor) processEvent(args *argsProcessEvent) argOutputProcessEvent {
    identifier := string(args.event.GetIdentifier())
    if _, ok := drwaCanonicalEventsMap[strings.ToLower(identifier)]; !ok {
        return argOutputProcessEvent{}
    }
}
```

```
// Verify event comes from authorised registry contract
// args.logAddress is the raw address bytes of the emitting contract
if dep.drwaRegistryAddress != "" {
    // hex-encode the raw log address for comparison
    emitterHex := hex.EncodeToString(args.logAddress)
    if emitterHex != dep.drwaRegistryAddress {
        log.Warn("drwaEventsProcessor: event from unauthorised contract",
            "identifier", identifier, "emitter", emitterHex, "expected",
            dep.drwaRegistryAddress)
        return argOutputProcessEvent{}
    }
}
// ... rest unchanged
}
```

Add registry address to config:

```
# config.toml
[config.drwa]
    registry-address = "" # hex-encoded address of authorised DRWA registry contract
```

Why This Fix Is Safe: If `drwaRegistryAddress` is empty, the check is skipped — backward compatible with existing deployments. If set, only events from the configured address are processed. No semantic change for legitimate events from the authorised registry.

Test Update Required: No existing tests need to be updated. All existing DRWA tests use mock events with no contract address verification. The fix only affects events from unauthorised contracts, which had no passing test.

Why This Fix Is Necessary: A compliance index that accepts events from any contract is not a compliance index — it is a public append-only log that any actor can write to.

Silence is worse than explicit failure because a spoofed `kycStatus=approved` record written by an unauthorised contract gives no indication to the compliance engineer or the regulatory officer that the record did not originate from the authorised DRWA registry.

Finding 8 — REAL FINDING — Unvalidated On-Chain Strings in Compliance Records in `drwaEventsProcessor.go` lines 288–506

Classification: - CWE: CWE-20 (Improper Input Validation) - CVSS v3.1 Score: **5.3 (Medium)** — AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:H/A:N - Severity: **Medium** - Fix Required: Yes - Runtime Impact: Arbitrary strings from on-chain event topics stored verbatim as `KYCStatus`, `AMLStatus`, `JurisdictionCode`, `EntityType`, `InvestorClass`, `RegistrationStatus`, `WhitePaperCID` — compliance consumers receive unvalidated attacker-controlled field values - Monitoring Impact: No field in the indexed document indicates whether a compliance field value was validated — spoofed values are indistinguishable from legitimate ones

Severity Note: Medium because the storage path uses `json.Marshal` via struct serialisation — there is no Elasticsearch injection risk. However, the compliance fields are stored verbatim from on-chain event topics with no length limit, no character validation, and no allowlist check. Combined with Finding 7 (no contract address verification), any contract can write arbitrary strings into KYC/AML status fields. Even with Finding 7 fixed, a compromised or buggy DRWA registry contract can write unexpected values into these fields.

What the Vulnerable Function Does:

`tryBuildIdentityRecord`, `tryBuildHolderComplianceRecord`, and `tryBuildTokenPolicyRecord` cast raw on-chain topic bytes directly to strings using `string(topics[N])` with no validation. The resulting strings are stored in compliance record structs which are then serialised to Elasticsearch via `json.Marshal`.

Call chain: `processEvent` → `tryBuildHolderComplianceRecord` → `string(topics[3])` assigned to `KYCStatus` → struct serialised via `json.Marshal` → written to `drwa-holder-compliance` index.

Where Does the Vulnerable Data Come From:

DRWA registry contract (or any contract — see Finding 7) → emits event with arbitrary topic bytes → `args.event.GetTopics()` returns raw `[][]byte` → `string(topics[N])` casts bytes to string with no validation → stored in compliance record → written to Elasticsearch.

Who Uses This Data and Why It Must Be Trusted:

1. **DevOps (Node Operator):** Monitors compliance field values for expected values. Breaks if arbitrary strings appear in `kycStatus` — monitoring alerts fire on unexpected values. Silent failure consequence: alert fatigue from unexpected values causes operators to disable compliance field monitoring.
 2. **Security (Compliance Engineer):** Relies on `kycStatus` and `amlStatus` containing only known values (`approved`, `pending`, `rejected`). Breaks if arbitrary strings are stored — downstream logic that switches on these values behaves unexpectedly. Silent failure consequence: a `kycStatus` value of `approved`; `DRWP` `TABLE` causes downstream string-parsing logic to fail silently.
 3. **Compliance (Regulatory Officer):** Must demonstrate that compliance field values conform to a defined schema. Breaks if arbitrary strings appear in regulatory reports. Silent failure consequence: regulatory filing contains unexpected field values — auditor flags the report as non-conformant.
 4. **On-Call Engineer:** Investigates unexpected compliance field values. Breaks because there is no way to distinguish a legitimate unexpected value from an attacker-injected one without replaying the block. Silent failure consequence: on-call cannot determine root cause without full block replay.
-

What an Attacker Can Do:

1. **KYC Status Spoofing:** Attacker emits `drwaHolderCompliance` with `topics[3] = "approved"` for a non-compliant address — stores a fake approval.
 - Consequence: Non-compliant holder appears KYC approved; regulated transfers proceed.
2. **Oversized Field DoS:** Attacker emits event with a 1MB `whitePaperCID` topic — indexer stores 1MB string in every token policy record.
 - Consequence: Elasticsearch document size limit exceeded; bulk request fails silently for that document.
3. **Downstream Logic Corruption:** Attacker stores a `kycStatus` value containing special characters that break downstream string-parsing logic in compliance dashboards.
 - Crafted input: `topics[3] = "approved\x00\x01\x02"`
 - Consequence: Compliance dashboard crashes or displays corrupted data when rendering the field.

4. **Jurisdiction Code Spoofing:** Attacker stores an invalid jurisdiction code (XX, --, INVALID) — compliance logic that validates jurisdiction codes rejects all transfers for that holder.

- Consequence: Legitimate holder is blocked from all regulated transfers due to an invalid jurisdiction code in their compliance record.

Why This Is Specific to This Feature:

DRWA compliance fields carry regulatory significance — `kycStatus=approved` is not just a string, it is a compliance decision that gates regulated token transfers. No other indexed field in the system has this property. Storing these fields without validation means the compliance gate can be influenced by any string that can be placed in an on-chain event topic.

The Fix:

BEFORE:

```
record.KYCStatus      = string(topics[3])
record.AMLStatus      = string(topics[4])
record.InvestorClass  = string(topics[5])
record.JurisdictionCode = string(topics[6])
```

AFTER:

```
var allowedComplianceStatuses = map[string]struct{}{
    "approved": {}, "pending": {}, "rejected": {}, "expired": {}, "": {},
}

func validateComplianceStatus(raw []byte) string {
    v := strings.TrimSpace(string(raw))
    if _, ok := allowedComplianceStatuses[strings.ToLower(v)]; ok {
        return v
    }
    log.Warn("drwaEventsProcessor: unrecognised compliance status", "value", v)
    return ""
}

func validateJurisdictionCode(raw []byte) string {
    v := strings.TrimSpace(string(raw))
    if matched, _ := regexp.MatchString(`^[A-Z]{2}$`, v); matched || v == "" {
        return v
    }
    log.Warn("drwaEventsProcessor: invalid jurisdiction code", "value", v)
    return ""
}

func validateFieldLength(raw []byte, maxLen int) string {
    v := strings.TrimSpace(string(raw))
    if len(v) > maxLen {
        log.Warn("drwaEventsProcessor: field too long, truncating", "len", len(v))
        return v[:maxLen]
    }
    return v
}
```

```
// In tryBuildHolderComplianceRecord:
record.KYCStatus      = validateComplianceStatus(topics[3])
record.AMLStatus      = validateComplianceStatus(topics[4])
record.InvestorClass  = validateFieldLength(topics[5], 64)
record.JurisdictionCode = validateJurisdictionCode(topics[6])
```

Why This Fix Is Safe: Validation functions return empty string for invalid values — downstream logic already handles empty compliance fields. No semantic change for valid inputs from the authorised registry.

Test Update Required: No existing tests need to be updated. All existing DRWA tests use valid topic values that pass validation unchanged.

Why This Fix Is Necessary: Compliance fields that accept arbitrary strings are not compliance fields — they are unvalidated text storage that any on-chain actor can write to.

Silence is worse than explicit failure because a `kycStatus` field containing an attacker-controlled string gives no indication to the compliance engineer or the regulatory officer that the value was not produced by the authorised DRWA registry.

Finding 9 — DRWA SAFE / HARDENING — Document ID Oversized Silent Bulk Drop in `serializeDrwa.go` line 127

Classification: Not a vulnerability — confirmed safe by code review. Hardening recommended. - Severity: Info - Fix Required: No (recommended)

What Was Reviewed:

`prepareDRWARecord` constructs Elasticsearch bulk API meta lines using `fmt.Sprintf` with on-chain values (`denialCode`, `Subject`, `TokenID`, `Auditor`) passed through `converters.JsonEscape`. `JsonEscape` uses `json.Marshal` which escapes all special characters — quotes, backslashes, and control characters are neutralised. The `_id` value cannot break out of the JSON string context. **This is not an injection vulnerability.**

```
// SAFE - JsonEscape uses json.Marshal
meta := []byte(fmt.Sprintf(
    `{ "index" : { "_index": "%s", "_id" : "%s" } }%s`,
    index,
    converters.JsonEscape(id), // json.Marshal escapes all special chars
    "\n",
))
```

The Hardening Issue:

Elasticsearch enforces a 512-byte limit on `_id` values. If a composed document ID (e.g. `txHash-denial-CODE-eventOrder`) exceeds 512 bytes, the bulk request fails silently for that document — no error is returned to the caller, the document is simply not indexed. A malicious on-chain actor who registers a token with a very long denial code can cause DRWA denial records to be silently dropped from the index.

Recommended Hardening:

```
func prepareDRWARecord(id string, index string, record any) ([]byte, []byte, error) {
    const maxIDLength = 400 // conservative limit below ES 512-byte max
    if len(id) > maxIDLength {
        log.Warn("prepareDRWARecord: document ID too long, truncating",
            "original_len", len(id), "max", maxIDLength)
        id = id[:maxIDLength]
    }
}
```

```

    serialized, err := json.Marshal(record)
    if err != nil {
        return nil, nil, err
    }
    meta := []byte(fmt.Sprintf(
        `{ "index" : { "_index": "%s", "_id" : "%s" } }%s`,
        index, converters.JsonEscape(id), "\n",
    ))
    return meta, serialized, nil
}

```

Finding 10 — REAL FINDING — xpack.security Disabled in `docker-compose.yml` line 7

Classification: - CWE: CWE-732 (Incorrect Permission Assignment for Critical Resource) - CVSS v3.1 Score: **6.5 (Medium)** — AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N - Severity: **Medium** - Fix Required: Yes - Runtime Impact: Elasticsearch runs with all security features disabled — no authentication, no TLS, no RBAC, no audit logging - Monitoring Impact: No audit log exists — all access to indexed data is invisible

Severity Note: Medium because this is the development docker-compose file. However it directly enables Finding 4 at the infrastructure level — `xpack.security.enabled=false` disables authentication, TLS, field-level security, and audit logging in a single line. If used as a staging or production template (common pattern), the entire cluster is exposed.

The Vulnerable Code:

```

# VULNERABLE
environment:
  - "xpack.security.enabled=false"  # disables ALL security features
ports:
  - "9200:9200"  # exposed on all interfaces
  - "9300:9300"  # transport port also exposed

```

What an Attacker Can Do:

1. Port scan finds 9200 open → `curl http://host:9200/drwa-holder-compliance/_search?size=10000` → all KYC/AML records exfiltrated with no credentials.
2. Open Kibana at port 5601 → full visual access to all indexed data, no login required.
3. `curl -X DELETE http://host:9200/drwa-holder-compliance` → entire compliance index deleted, no audit trail.
4. Inject fake compliance records directly via ES API, bypassing all indexer validation.

The Fix:

```

# FIXED
# WARNING: LOCAL DEVELOPMENT ONLY – do not use for staging or production
environment:
  - "xpack.security.enabled=true"
  - "xpack.security.audit.enabled=true"
  - "ELASTIC_PASSWORD=${ES_ADMIN_PASSWORD}"
ports:
  - "127.0.0.1:9200:9200"  # localhost only
  - "127.0.0.1:9300:9300"  # localhost only

```

```
# .env – add to .gitignore
ES_ADMIN_PASSWORD=change_me_strong_password
```

Why This Fix Is Safe: Existing local development workflows continue to work — only the binding address and security settings change. Operators who need external access configure their own port forwarding explicitly.

Test Update Required: Integration tests that use docker-compose must set `ELASTIC_PASSWORD` and configure the ES client with credentials. No unit tests are affected.

Finding 11 — REAL FINDING — Plaintext Credential Structure in `config.json` lines 3–5

Classification: - CWE: CWE-312 (Cleartext Storage of Sensitive Information) - CVSS v3.1 Score: **5.5 (Medium)** — AV:L/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N - Severity: **Medium** - Fix Required: Yes - Runtime Impact: Elasticsearch credentials stored in plaintext on disk — any process with read access to the file extracts credentials silently - Monitoring Impact: No way to detect credential theft from the config file

Severity Note: Medium because the file is committed with empty values. However the structure actively invites operators to fill in real credentials. If populated and committed — a very common mistake — credentials are permanently in git history. The `accounts-balance-checker` tool has direct read/write access to the Elasticsearch cluster including all DRWA compliance indices.

The Vulnerable Code:

```
{
  "elasticsearch": {
    "url": "",
    "username": "", // operator fills in real credentials here
    "password": "" // stored in plaintext on disk
  }
}
```

What an Attacker Can Do:

1. `git log --all -p -- config.json` → credentials visible in every clone if ever committed.
2. Any process on the same host with file read access extracts credentials silently.
3. Credentials used to access production Elasticsearch — all DRWA compliance data exfiltrated.
4. `--repair` flag + stolen credentials → attacker overwrites account balances in the index.

The Fix:

```
// config.json – safe to commit, no credential fields
{
  "elasticsearch": { "url": "https://localhost:9200" },
  "proxy": { "url": "", "parallel-requests": 40 }
}
```

```
// main.go – load credentials from environment only
cfg.Elasticsearch.Username = requireEnv("ES_USERNAME")
cfg.Elasticsearch.Password = requireEnv("ES_PASSWORD")

func requireEnv(key string) string {
    val := os.Getenv(key)
```

```

    if val == "" {
        log.Error("required environment variable not set", "key", key)
        os.Exit(1)
    }
    return val
}

```

Apply the same fix to `tools/clusters-checker/cmd/checker/config.toml` which has the same `user` and `password` fields.

Why This Fix Is Safe: No runtime change for correctly configured deployments. `requireEnv` fails fast at startup — no silent unauthenticated deployment possible.

Test Update Required: No existing tests need to be updated.

Finding 12 — CODE QUALITY — 1-Second Graceful Shutdown in `httpServer.go` line 50

Classification: - CWE: CWE-400 (Uncontrolled Resource Consumption) - Severity: **Info** - Fix Required: No (recommended)

The Vulnerable Code:

```

func (h *httpServer) Close() error {
    ctx, cancel := context.WithTimeout(context.Background(), time.Second) // 1 second
    defer cancel()
    return h.server.Shutdown(ctx)
}

```

What is wrong: 1 second is too short for any non-trivial in-flight request to complete. During a rolling restart, active Prometheus scrapes or metrics polls are forcibly terminated — monitoring gaps occur at every restart.

The Fix:

```

const shutdownTimeout = 30 * time.Second

func (h *httpServer) Close() error {
    ctx, cancel := context.WithTimeout(context.Background(), shutdownTimeout)
    defer cancel()
    return h.server.Shutdown(ctx)
}

```

Finding 13 — FALSE POSITIVE — Empty Credentials Passed to ES Client in

`indexerFactory.go` lines 110–112

What the Scanner Flagged: Empty `Username` and `Password` strings passed to `elasticsearch.Config` — flagged as potential credential leak or misconfiguration.

Why It Is a False Positive:

The empty strings are the default template values from `prefs.toml`. The Elasticsearch Go client correctly omits the `Authorization` header when credentials are empty strings — it does not send `Authorization: Basic og==` (base64 of `:`). The actual security issue is that the application starts successfully with empty credentials (Finding 4) — but the act of passing empty strings to the client constructor is not itself a vulnerability. The client behaves correctly; the vulnerability is the missing startup validation.

```
// indexerFactory.go lines 110-112 – not a bug in itself
argsEsClient := elasticsearch.Config{
    Addresses: []string{args.Url},
    Username:  args.UserName, // "" – client omits Authorization header
    Password:  args.Password, // "" – correct behaviour
}
```

Action required: None at this location. Fix is in `wsIndexerFactory.go` — add startup validation (see Finding 4).

SECTION 3 — FALSE POSITIVES

`drwaCanonicalEventsMap` Allowlist — Not a Vulnerability

The explicit allowlist of DRWA event identifiers using exact lowercase matching is a correct and secure design. Prefix matching was intentionally avoided — the comment in the code states this explicitly. Any event whose identifier is not in the map is silently ignored. This is not a vulnerability.

`normalizeDRWADenialCode` Verbatim Preservation — Not a Vulnerability

`normalizeDRWADenialCode` preserves unrecognised denial codes verbatim after trimming. This is an intentional design decision to avoid dropping evidence. The value is stored via `json.Marshal` through the struct serialisation path — there is no injection risk. This is not a vulnerability.

WebSocket Connection — Not a Vulnerability

The WebSocket host is configured from `prefs.toml` (operator-controlled). The trust boundary is the WebSocket peer (the mx-chain-go node). Compromise of the node is outside the threat model of the indexer itself.

SECTION 4 — FEATURE SECURITY ASSESSMENT

Feature Area	Status	Notes
Painless Script Construction	VULNERABLE	<code>PrepareNFTUpdateData</code> embeds on-chain <code>newMetadata</code> into Painless source via <code>fmt.Sprintf</code> . Fix: static script + <code>json.Marshal</code> params (Finding 1).
DRWA Revert Path	VULNERABLE	<code>removeFromIndexByBlockHashAndShardID</code> embeds <code>blockHash</code> via <code>fmt.Sprintf</code> . Fix: <code>json.Marshal</code> (Finding 2).
DRWA Finalize Path	VULNERABLE	<code>prepareDRWAFinalizedBlockQuery</code> embeds <code>blockHash</code> via <code>fmt.Sprintf</code> . Fix: <code>json.Marshal</code> + error return (Finding 3).
Elasticsearch Authentication	VULNERABLE	

Feature Area	Status	Notes
		Empty credentials accepted at startup; cluster connects unauthenticated. Fix: fail-fast validation + env vars (Finding 4).
CORS Policy	VULNERABLE	<code>AllowAllOrigins=true</code> with <code>Authorization</code> header. Fix: origin whitelist + remove <code>Authorization</code> (Finding 5).
Balance Checker Queries	VULNERABLE	<code>queryGetLastTxForToken</code> embeds on-chain identifier via <code>fmt.Sprintf</code> . Fix: <code>json.Marshal</code> (Finding 6).
DRWA Event Source Verification	VULNERABLE	No contract address check — any contract can emit canonical DRWA events. Fix: registry address validation (Finding 7).
DRWA Compliance Field Validation	VULNERABLE	Raw on-chain topic bytes stored verbatim in KYC/AML fields. Fix: allowlist validation (Finding 8).
DRWA Document ID Safety	SAFE	<code>JsonEscape</code> uses <code>json.Marshal</code> — injection impossible. Hardening: 400-byte ID length cap (Finding 9).
Elasticsearch Infrastructure	VULNERABLE	<code>xpack.security.enabled=false</code> in docker-compose. Fix: enable security, bind to localhost (Finding 10).
Credential Storage	VULNERABLE	Config file structure invites plaintext credential commits. Fix: env vars only (Finding 11).
Graceful Shutdown	PARTIAL	1-second timeout drops in-flight requests. Fix: 30-second timeout (Finding 12).
DRWA Canonical Event Allowlist	SECURE	Exact lowercase matching — prefix attacks impossible. No fix needed.
DRWA Denial Code Normalisation	SECURE	Verbatim preservation via <code>json.Marshal</code> — no injection risk. No fix needed.
Concurrency	SECURE	<code>elasticProcessor</code> uses <code>sync.RWMutex</code> for <code>importDB</code> . <code>DrwaCounterSet</code> uses <code>sync.Mutex</code> . No data races detected.

SECTION 5 — ACTION PLAN

Priority	Action	File	Line	Effort
P0 — Critical	Replace <code>fmt.Sprintf</code> with static script + <code>json.Marshal</code> params in <code>PrepareNFTUpdateData</code>	<code>process/elasticproc/converters/tokenMetadata.go</code>	160	1 hour
P0 — Critical	Replace <code>fmt.Sprintf</code> with <code>json.Marshal</code> in	<code>process/elasticproc/</code>	432	30 min

Priority	Action	File	Line	Effort
	<code>removeFromIndexByBlockHashAndShardID</code>	<code>elasticProcessor.go</code>		
P0 — Critical	Replace <code>fmt.Sprintf</code> with <code>json.Marshal</code> in <code>preparedDRWAFinalizeDBlockQuery</code>	<code>process/elasticproc/elasticProcessor.go</code>	443	30 min
P0 — Critical	Add fail-fast credential validation at startup; load from env vars	<code>factory/wsIndexerFactory.go</code>	—	30 min
P1 — High	Add contract address verification in <code>drwaEventsProcessor.processEvent</code>	<code>process/elasticproc/logsevents/drwaEventsProcessor.go</code>	88	1 hour
P1 — High	Replace <code>AllowAllOrigins=true</code> with origin whitelist; remove Authorization header	<code>api/gin/webServer.go</code>	61	30 min
P1 — High	Replace <code>fmt.Sprintf</code> with <code>json.Marshal</code> in <code>queryGetLastTxForToken</code> and <code>queryGetLastOperationForAddress</code>	<code>tools/accounts-balance-checker/pkg/check/query.go</code>	48	30 min
P2 — Medium	Add allowlist validation for <code>KYCStatus</code> , <code>AMLStatus</code> , <code>JurisdictionCode</code> , <code>InvestorClass</code>	<code>process/elasticproc/logsevents/drwaEventsProcessor.go</code>	288–506	1 hour
P2 — Medium	Enable <code>xpack.security</code> , bind ports to localhost, add <code>.env</code>	<code>docker-compose.yml</code>	7	30 min
P2 — Medium	Remove credential fields from <code>config.json</code> ; load from env vars	<code>tools/accounts-balance-checker/cmd/balance-checker/config.json</code>	3	30 min
P3 — Low	Add 400-byte document ID length cap in <code>preparedDRWARecord</code>	<code>process/elasticproc/logsevents/serializeDrwa.go</code>	121	15 min
P3 — Low	Increase graceful shutdown timeout from 1s to 30s	<code>api/gin/httpServer.go</code>	50	5 min

Test Impact Summary

Fix	File	Test Action Required
Finding 1 — Painless Injection	process/ elasticproc/ converters/ tokenMetadata_test.go	No change — semantic outcome identical for valid inputs
Finding 2 — Revert Query Injection	process/ elasticproc/ elasticProcessor_test.go	No change — query semantics identical for valid hex hashes
Finding 3 — Finalize Query Injection	process/ elasticproc/ elasticProcessor_test.go	Update callers of <code>prepareDRWAFinalizedBlockQuery</code> to handle <code>error</code> return
Finding 4 — Missing Auth	Integration tests	Set <code>ES_USERNAME</code> and <code>ES_PASSWORD</code> in test environment
Finding 5 — CORS	api/gin/ webServer_test.go	No change — CORS not tested in current suite
Finding 6 — Balance Checker Injection	tools/accounts- balance-checker/ pkg/check/ query_test.go	Update callers to handle <code>error</code> return from fixed functions
Finding 7 — Contract Address	process/ elasticproc/ logsevents/ drwaEventsProcessor_test.go	Add test for event from unauthorised contract — expect empty result
Finding 8 — Field Validation	process/ elasticproc/ logsevents/ drwaEventsProcessor_test.go	Add tests for invalid KYC/AML values — expect empty string stored
Finding 10 — docker-compose	integrationtests/ /	Set <code>ELASTIC_PASSWORD</code> in CI environment
Finding 12 — Shutdown Timeout	api/gin/ httpServer_test.go	No change

SECTION 6 — FINAL DECISION

- **Finding 1** (Painless Script Injection): **NOT FIXED** — High severity. `newMetadata` from on-chain NFT attributes embedded raw into Painless source via `fmt.Sprintf`. Fix: static script + `json.Marshal` params in `tokenMetadata.go` line 160.

- **Finding 2** (DRWA Revert Query Injection): **NOT FIXED** — High severity. `blockHash` embedded raw into delete-by-query via `fmt.Sprintf` — can wipe all six DRWA compliance indices on a crafted revert message. Fix: `json.Marshal` in `elasticProcessor.go` line 432.
- **Finding 3** (DRWA Finalize Query Injection): **NOT FIXED** — High severity. `blockHash` embedded raw into update-by-query via `fmt.Sprintf` — can mass-finalize all DRWA records on a crafted finalize message. Fix: `json.Marshal` + error return in `elasticProcessor.go` line 443.
- **Finding 4** (Missing ES Authentication): **NOT FIXED** — High severity. Empty credentials accepted at startup; entire cluster accessible without authentication. Fix: fail-fast validation + env vars in `wsIndexerFactory.go`.
- **Finding 5** (CORS AllowAllOrigins): **NOT FIXED** — High severity. Any website can read indexer metrics from a victim's browser; Authorization header pre-authorized for future endpoints. Fix: origin whitelist in `webServer.go` line 61.
- **Finding 6** (Balance Checker Query Injection): **NOT FIXED** — Medium severity. On-chain token identifier embedded raw into ES query — can expand query scope to entire index. Fix: `json.Marshal` in `query.go` line 48.
- **Finding 7** (No Contract Address Verification): **NOT FIXED** — Medium severity. Any contract can emit canonical DRWA events and write spoofed compliance records. Fix: registry address validation in `drwaEventsProcessor.go` line 88.
- **Finding 8** (Unvalidated Compliance Strings): **NOT FIXED** — Medium severity. Raw on-chain bytes stored verbatim in KYC/AML fields with no allowlist check. Fix: validation functions in `drwaEventsProcessor.go` lines 288–506.
- **Finding 9** (Document ID Length): **DISMISSED** — Info. `JsonEscape` is safe. Hardening recommended: 400-byte ID cap in `serializeDrwa.go`.
- **Finding 10** (docker-compose Security): **NOT FIXED** — Medium severity. `xpack.security.enabled=false` disables all ES security features. Fix: enable security, bind to localhost in `docker-compose.yml`.
- **Finding 11** (Plaintext Credentials): **NOT FIXED** — Medium severity. Config file structure invites credential commits. Fix: env vars only in `config.json` and `main.go`.
- **Finding 12** (Shutdown Timeout): **DISMISSED** — Info. 1-second timeout is too short but has no security impact. Fix recommended: 30 seconds in `httpServer.go`.
- **Finding 13** (Empty Credentials to ES Client): **DISMISSED** — False positive. ES client correctly omits Authorization header for empty credentials. Root cause is Finding 4.

Fixing Findings 1 + 2 + 3 alone = Injection-Safe — no on-chain actor or compromised WebSocket peer can corrupt the DRWA compliance index via query or script injection.

Fixing Finding 4 alone = Authentication-Enforced — the Elasticsearch cluster cannot be accessed without credentials.

Fixing Findings 1 + 2 + 3 + 4 = Secure Core — the compliance pipeline is injection-safe and access-controlled.

Fixing everything (1 through 11) = Perfect at Peak — zero known security issues, validated compliance fields, authenticated infrastructure, origin-restricted API, and no credential exposure vectors.

SECTION 7 — DRWA FLOW INTEGRITY GAP

This section covers a gap in the report that is specific to the indexer's role in the full DRWA compliance flow. It is not a code bug — it is a design gap between what the indexer promises and what it can guarantee.

DRWA Flow Gap — REAL FINDING — No Fallback for Unfinalized DRWA Records

Classification: - CWE: CWE-691 (Insufficient Control Flow Management) - CVSS v3.1 Score: **5.3 (Medium)** — AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:H/A:N - Severity: **Medium** - Fix Required: Yes - Runtime Impact: DRWA compliance records remain permanently in `isFinalized=false` state if the `FinalizedBlock` WebSocket message is dropped, delayed, or never sent — downstream compliance consumers cannot distinguish “pending finality” from “finality message was lost” - Monitoring Impact: No metric, no alert, and no timeout exists to detect records that have been in `isFinalized=false` state longer than the expected finality window

The DRWA Flow Context:

The indexer sits at the end of the DRWA compliance pipeline:

```
On-chain DRWA contract emits event
  → mx-chain-go node processes block
  → OutputBlock delivered to indexer via WebSocket
  → indexer writes DRWA records with IsFinalized=false
  → block reaches finality on-chain
  → mx-chain-go node sends FinalizedBlock message via WebSocket
  → indexer sets isFinalized=true on matching records
  → downstream consumers treat isFinalized=true as confirmed
```

The vmcommon repo (mx-chain-vm-common-go) enforces compliance at transfer time using in-memory trie state. The indexer is the **only persistent audit trail** — it is what makes compliance enforcement auditable, queryable, and reportable to regulators.

The Gap:

Every DRWA record is written with `IsFinalized: false` hardcoded:

```
// drwaEventsProcessor.go - every single DRWA record builder
record := &data.DrwaIdentityRecord{
    IsFinalized: false, // hardcoded - only FinalizedBlock can set this to true
    ...
}
```

`isFinalized` is set to `true` only by `prepareDRWAFinalizedBlockQuery` called from `FinalizedBlock`. There is no: - Timeout after which unfinalized records are automatically finalized - Metric tracking how many records have been in `isFinalized=false` state for longer than the expected finality window (~2 rounds on MultiversX) - Alert when a record remains unfinalized beyond the expected window - Fallback that marks records as finalized if the `FinalizedBlock` message is never received

What This Means for the DRWA Flow:

If the WebSocket connection drops after `saveBlock` but before `FinalizedBlock` is received, all DRWA records from that block remain permanently in `isFinalized=false` state. The indexer has no way to recover — it cannot re-request the `FinalizedBlock` message, and it cannot determine from the block data alone whether the block was finalized.

Downstream compliance consumers that filter on `isFinalized=true` will never see these records. Downstream consumers that include `isFinalized=false` records cannot distinguish “pending finality” from “finality message was lost.”

Who Is Affected:

1. **Compliance dashboards** that show only finalized compliance events — will permanently miss events from blocks whose finalize message was dropped
2. **Regulatory reporting tools** that require `isFinalized=true` — will produce incomplete reports
3. **Audit trail consumers** — cannot determine whether a gap in finalized records means “no events occurred” or “finalize messages were lost”

The Fix:

Step 1 — add a metric tracking the age of the oldest unfinalized DRWA record:

```
// In statusMetrics or a new DRWA-specific metrics component
func (sm *StatusMetrics) RecordDRWAUnfinalizedRecordAge(blockRound uint64, currentRound uint64)
{
    age := currentRound - blockRound
    if age > drwaExpectedFinalityRounds {
        sm.AddIndexingData(ArgsAddIndexingData{
            Topic: "drwa_unfinalized_record_stale",
        })
    }
}
```

Step 2 — add a periodic reconciliation job that marks records as finalized if their `blockRound` is older than the finality window and no revert has been received:

```
// New component: drwaFinalityReconciler
// Runs every N rounds, queries ES for isFinalized=false records
// older than expectedFinalityRounds, marks them finalized
func (r *drwaFinalityReconciler) reconcile(currentRound uint64) error {
    staleThreshold := currentRound - drwaExpectedFinalityRounds
    query := map[string]interface{}{
        "query": map[string]interface{}{
            "bool": map[string]interface{}{
                "must": []interface{}{
                    map[string]interface{}{"term": map[string]interface{}{"isFinalized": false}},
                },
            },
            "range": map[string]interface{}{
                "blockRound": map[string]interface{}{"lt": staleThreshold},
            },
        },
    },
    "script": map[string]interface{}{
        "source": "ctx._source.isFinalized = true",
        "lang": "painless",
    },
}
// execute update_by_query against all DRWA indices
}
```

Step 3 — add an alert threshold in the monitoring configuration:

```
# prometheus alert rule
- alert: DRWARecordUnfinalizedTooLong
  expr: drwa_unfinalized_record_stale > 0
  for: 5m
  annotations:
    summary: "DRWA compliance records have been unfinalized for longer than expected finality window"
```

Why This Gap Matters for the DRWA Flow:

The vmcommon repo enforces compliance at transfer time — if a transfer is denied, it is denied on-chain regardless of what the indexer says. But the indexer is the only system that provides: - A queryable history of all compliance decisions - Evidence that a specific holder was KYC/AML approved at a specific block - Proof that a transfer denial occurred at a specific time

If `isFinalized=false` records are never finalized due to a dropped WebSocket message, the compliance audit trail has permanent gaps that cannot be explained to regulators. The on-chain state is correct; the indexed audit trail is incomplete.

Why This Is Specific to This Repo:

The vmcommon repo has no concept of `isFinalized` — it enforces compliance in real time using trie state. The `isFinalized` flag exists only in the indexer. The indexer is the only component in the entire DRWA system that can have permanently unfinalized compliance records. This gap does not exist in the vmcommon repo — it is unique to the indexer's role as the compliance audit trail.